此为临时预览链接，将在短期内失效

06 领域本体建模实战：范围界定、能力问题与建模原则

陈乔 通往AI之桥 2026年10月25日 20:31

先问清本体要解决什么，再决定世界应该如何被描述

通往AI之桥 | 本体论在 AI 工程中的实践系列 · 06

上一篇，我们讨论了如何从业务文档、SQL、API、事件日志和专家经验中发现候选概念。

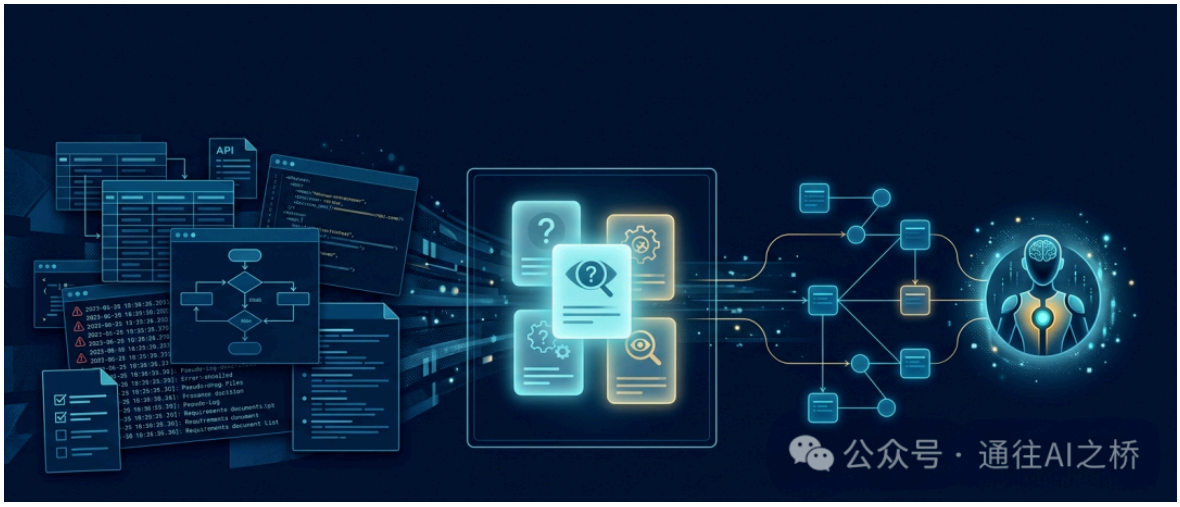
但发现了一堆术语，并不意味着本体已经设计完成。把“客户、订单、组件、版本、测试、发布”画成一张关系图，也不意味着这张图真的有用。

真正进入建模阶段，最先要做的不是选择 RDF 还是属性图，也不是决定用哪个工具画图，而是回答三个问题：

1. **这份本体到底服务哪个业务范围?**
2. **它必须帮助我们回答哪些问题?**
3. **怎样判断模型足够好，而不是越建越大?**

如果这三个问题没有答案，本体项目很容易陷入两种极端：一种是过度建模，试图一次性描述整个企业；另一种是把数据库表、系统菜单和现有流程原样搬进来，最后得到的只是另一份技术结构图。

> **本体建模不是把世界描述得越完整越好，而是用足够清晰、可验证、可维护的概念和关系，支撑一组明确的业务问题。**



【图 1 | 从庞杂的企业系统中，收敛出一份边界明确、可验证的领域本体】

此为临时预览链接，将在短期内失效

假设我们正在为一个 AI 软件工厂设计领域本体。这个工厂能够接收需求，生产软件组件和产品，执行开发、测试、安全检查、发布、部署与运维升级。

这里面可以建模的东西太多了：客户、合同、预算、需求、产品、组件、代码仓库、提交、构建、测试、漏洞、发布、集群、服务器、部署、工单、账单、采购、供应商……如果把所有名词都列进第一版，模型很快就会膨胀，讨论也会失去焦点。

所以，第一步是给本体写一份简短的“范围说明书”。它至少应包含以下内容：

要明确的事项	示例	为什么重要
业务目标	判断软件版本是否具备发布条件，并识别漏洞影响范围	避免建模目的过于宽泛
主要使用者	发布负责人、安全 Agent、测试 Agent、产品负责人	让模型贴近真实决策与工作任务
建模范围	产品、组件、版本、构建、测试结果、漏洞、发布、部署	只纳入当前问题必要的概念
数据边界	代码仓库、制品库、测试平台、漏洞扫描器、部署平台	明确本体如何连接现有系统
本期不做什么	不重建财务总账，不建模所有人力资源与客户合同细节	防止范围无限扩张
成功标准	可以追溯构建来源、判断发布缺项、查询漏洞影响范围	让项目结果可验收

这份说明书不是形式文件，而是帮助团队做取舍的依据。后来有人提出新增“采购订单”或“供应商评价”概念时，可以先问：它是否影响本期要解决的问题？如果没有，就可以放到后续范围，而不是立即加入核心模型。

三种边界要分别考虑

业务边界决定本体描述哪一片现实世界。例如，本期关注软件产品发布，而非软件公司的全部经营活动。

语义边界决定哪些概念由这份本体负责定义。例如，“组件版本”与“产品版本”应明确区分；“构建完成”与“发布批准”不是同一个状态。

系统边界决定本体需要连接哪些数据源和服务。例如，漏洞明细由安全扫描平台维护，本体不一定复制所有扫描数据，只要能以稳定标识关联并解释其含义即可。

此为临时预览链接，将在短期内失效

覆盖业务领域的全部知识，只需要覆盖已声明的用途。

二、用能力问题检验本体：先写问题，再画模型

本体工程中常用一种方法：**能力问题（Competency Questions，CQ）**。它指一组本体预期能够回答的问题，用来帮助界定范围、指导建模，并检查模型是否足以支撑目标用途。相关方法在 Grüninger 与 Fox 的本体工程研究中已有系统讨论。[2]

以软件工厂为例，先不要急着定义类和关系，而是写下业务方希望系统能够回答的问题。

第一组：追溯性问题

- 某个产品版本由哪些需求驱动?
- 该版本包含哪些组件及其版本?
- 某次构建使用了哪个代码提交、哪些依赖版本?
- 当前客户环境实际部署了哪个产品版本和制品?

第二组：质量与安全问题

- 某个候选版本有哪些测试通过、失败或尚未执行?
- 哪些未关闭的漏洞会影响该版本?
- 如果某个基础组件存在高危漏洞，哪些产品版本和客户环境可能受影响?
- 一个发布申请缺少哪些必要证据或审批?

第三组：变更影响问题

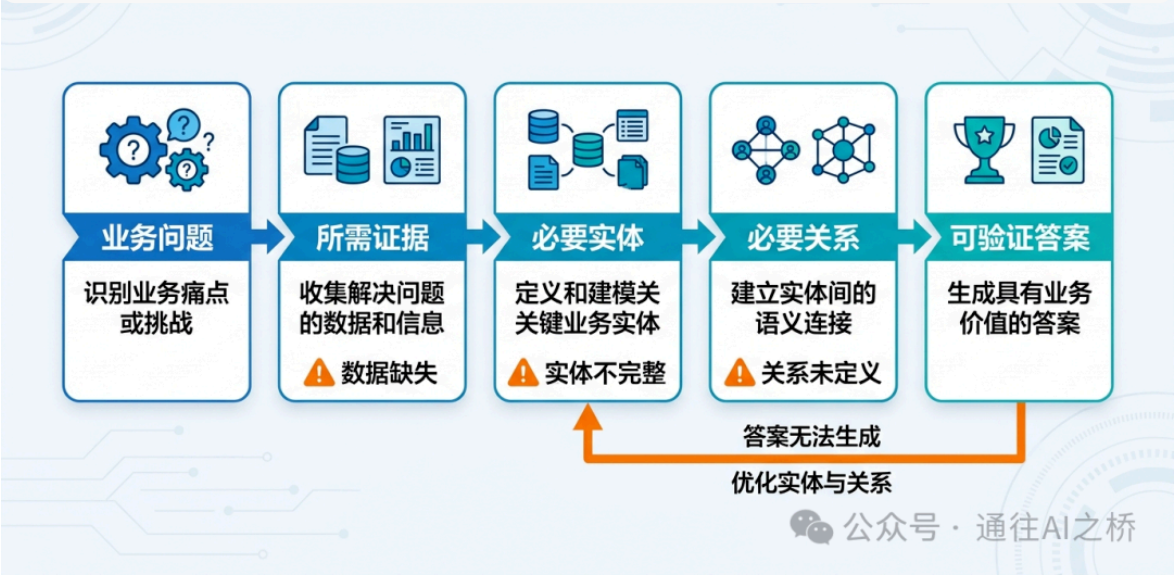
- 升级组件版本后，哪些产品需要回归测试?
- 某个 API 契约发生变更，会影响哪些调用方?
- 如果部署失败，如何定位到制品、配置、构建和代码变更?

接下来，把每个问题转成“所需信息清单”。例如，要回答“哪些客户环境受到某漏洞影响”，至少需要：漏洞影响组件版本、产品版本包含组件版本、客户环境部署产品版本。缺少其中任何一段关系，都可能无法可靠地回答问题。

这给建模带来了一个非常实用的检验标准：

> 如果一个能力问题无法回答，先检查模型缺少哪些概念、关系或数据映射；如果一个概念和关系长期不能帮助回答任何已声明的问题，也应该重新评估它是否属于当前范围。

此为临时预览链接，将在短期内失效



【图 2｜能力问题反推模型】

能力问题要具体到什么程度?

不够好的问题是：“我们需要支持软件质量管理。”这句话太宽泛，无法确定需要哪些概念，也不能写出验收测试。

更好的问题是：“给定一个候选产品版本，系统能否列出该版本涉及的组件版本、每个组件的测试状态，以及所有尚未处置的高危漏洞？”

再进一步，可以把它变成可执行的测试场景：提供一个已知的产品版本、组件依赖、测试结果和漏洞清单，检查系统是否返回预期影响范围与缺失项。

能力问题既可以是自然语言，也可以逐渐转化为查询、规则、自动化测试或验收用例。并非每个问题都必须形式化到逻辑公式，但至少要让业务人员和工程师对“什么算答对”达成一致。

三、建模原则一：粒度要适中，不要把所有东西都建成一个类

本体需要足够细，才能表达真实业务差异；也需要足够粗，避免模型复杂到无法维护。粒度是本体设计中最容易争论的部分之一。

以软件发布为例，下面这几个概念看起来相似，但通常应当区分：

- **Product（产品）**：对外提供的可交付软件。
- **ProductVersion（产品版本）**：产品在某一版本标识下的可交付状态。
- **Component（组件）**：可被多个产品复用的软件组成部分。
- **ComponentVersion（组件版本）**：具有明确版本标识和依赖的组件发布物。
- **Build（构建）**：一次构建执行及其结果。

此为临时预览链接，将在短期内失效

- Deployment（部署）：将某个制品部署到目标环境的执行或记录。

为什么不把这些统称为 SoftwareItem？因为它们有不同的身份、生命周期、属性和关系。一个产品版本可以有多次构建；一次构建可能生成多个制品；同一个制品可以被部署到多个环境；一次发布也未必等同于一次部署。

如果把它们合并成一个模糊的“软件对象”，就很难回答构建追溯、部署状态与版本影响等问题。

另一方面，也不能无限细分。例如，当前仅用于高层影响分析时，不一定需要建模源文件中的每个函数、每个代码行或每个构建日志字段。可以先保留提交、构建、制品与组件版本等关键层次，只有当具体问题确实需要更细粒度信息时再扩展。

判断粒度是否合适，可以问三个问题：

1. 这些对象是否有不同的身份或生命周期？
2. 它们是否有不同的关键属性、关系或约束？
3. 合并之后，会不会让某个能力问题无法正确回答？

如果前两个问题的答案为“是”，或第三个问题出现明显风险，就应认真考虑拆分概念。

四、建模原则二：命名要明确，定义比名称更重要

本体名称看起来只是文档问题，实际上会影响人、模型、查询和 API 对概念的理解。

例如，属性叫 status 太宽泛。它可能表示构建状态、测试状态、审批状态、发布状态或部署状态。更清晰的设计可以使用 buildStatus、testStatus、approvalStatus、deploymentStatus，或者将状态放在不同类型的对象上。

同样，关系名称应尽量表达明确方向：

- `productVersion includes componentVersion`
- `build produces artifact`
- `testRun validates artifact`
- `deployment deploys artifact to environment`
- `finding affects componentVersion`

相比 relatedTo、associatedWith 这类泛化关系，清晰的关系名更容易被理解、查询和校验。

建议为每个核心概念建立最小定义卡片：

此为临时预览链接，将在短期内失效

规范名称	Artifact
中文名称	软件制品
定义	由构建或打包过程产生，可存储、签名、扫描、分发或部署的软件交付对象
唯一标识	制品坐标、仓库地址加版本/摘要等稳定标识
关键属性	类型、摘要、生成时间、签名状态、来源构建
关键关系	由 Build 产生；属于某个产品版本；被 Deployment 部署
不包括	发布审批记录本身；部署动作本身
来源与责任人	制品库元数据；产品工程团队
示例与反例	示例：容器镜像或发布包；反例：仅表示“发布成功”的状态字符串

这里最重要的是“不包括”这一项。清楚说明一个概念不是什么，往往和说明它是什么同样重要。

五、建模原则三：复用已有本体，但不要盲目照搬

本体工程不需要每次从零开始。对于时间、组织、人员、软件标识、地理位置、产品分类等常见概念，可以评估是否复用已有标准、本体或行业词汇。复用有助于一致性和互操作，但不同来源的粒度和语义承诺未必一致。[1][3]

复用时应逐项确认：

- **语义是否一致？** 名称相同不代表定义相同。
- **粒度是否适合？** 外部本体可能过于通用，也可能过于专业。
- **维护状态是否合适？** 需要考虑版本、维护者和兼容性。
- **许可是否允许？** 商业产品应核对使用和再分发许可。
- **是否值得引入完整依赖？** 有时只需映射少量概念，而不是导入整个本体。

例如，可以复用通用的组织、标识和时间表示方式，但软件工厂的 Build、Artifact、Release 和 Deployment 等概念，仍需要结合自身生产流程明确界定。

不要为了显示技术先进而引入复杂的外部本体。如果它无法解决实际能力问题，反而增加理解、版本兼容和维护成本，就不值得复用。

六、建模原则四：模块化，让本体可分工、可演进

当本体变大后，可以按业务边界拆分为相互关联的模块。例如软件工厂可划分为：

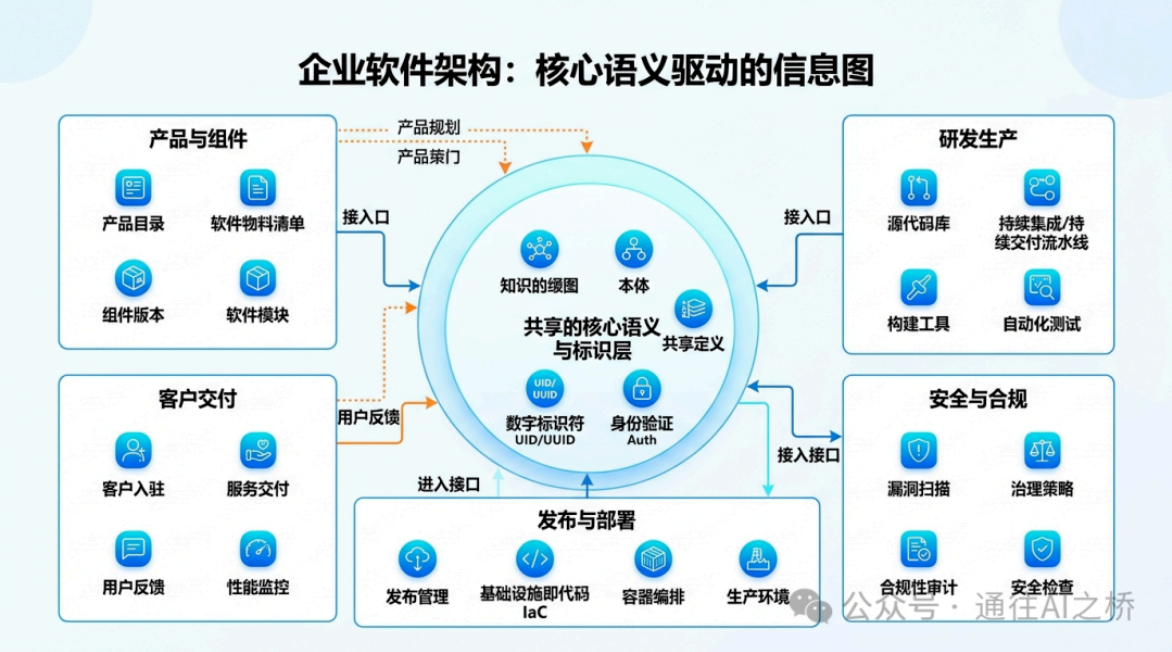
- **产品与组件模块**：产品、产品版本、组件、组件版本和依赖。

此为临时预览链接，将在短期内失效

- **发布与部署模块**：发布项、部署、运行环境和回滚。
- **客户交付模块**：客户、合同范围、配置差异和客户环境。

模块化的目标不是单纯把文件拆开，而是明确每个模块的责任边界、核心概念与对外关系。一个模块可以引用另一个模块定义的概念，但不应各自创建不同含义的同名对象。

例如，“组件版本”应由产品与组件模块定义；安全模块引用组件版本并表达漏洞影响；部署模块引用产品制品和环境。这样安全团队可以维护漏洞语义，发布团队可以维护发布流程，同时保持核心概念一致。



【图 3｜模块化领域本体架构】

七、建模原则五：概念模型与执行流程要分层

本体通常用于描述对象、关系及其语义约束；工作流用于组织任务顺序、状态转换、超时、重试、审批和执行动作。二者可以协同，但不应混成一件事。

例如，Release 是发布项，ReleaseApproved 可以是审批通过的状态或事件；“审批发布”则是一项动作，它由某个角色在满足条件时执行，并可能产生审批记录和状态变化。

可以用一个简单的分层理解：

- **本体层**：定义 ProductVersion、Build、TestRun、Finding、Release、Deployment 等概念及关系。
- **规则/约束层**：定义版本发布必须满足哪些条件，例如必需测试通过、没有未处理的阻断级风险。
- **工作流层**：负责按顺序运行测试、收集扫描结果、发起审批、批准或拒绝发布。

此为临时预览链接，将在短期内失效

本体可以帮助统一每一层对对象与关系的理解，但不能单独替代工作流引擎、策略执行点、权限系统或真实的工程测试。

尤其需要注意：**“模型里写了这条规则”不等于“生产系统已经强制执行这条规则”。**必须将规则映射到实际的校验器、审批策略和部署门禁，并用测试证明它们有效。

八、四种常见反模式：看起来建了模型，实际没有解决问题

反模式一：把数据库表直接复制成类

数据库表服务于某个应用的存储和实现；本体类服务于业务概念和跨系统语义。二者可能有映射关系，但不是一一对应。一个业务概念可能分散在多张表里，也可能因为历史原因在不同系统中采用不同结构。

建议：先从能力问题与业务定义出发，再把领域概念映射到表、视图、API 或事件；不要让物理结构直接决定语义模型。

反模式二：把每一个业务名词都建成类

“审批中”“已发布”“待处理”通常更适合作为状态值、状态概念或事件，而不是自动变成独立的业务类。究竟采用枚举、状态实体、事件还是类，需要依据身份、生命周期和关系来判断。

建议：在建类之前先问：这个词是否代表具有独立身份的对象？它是否有自己的关系和生命周期？还是只在描述另一个对象的状态？

反模式三：把工作流当成本体

“先构建，再测试，再审批，最后部署”描述的是流程顺序。流程中的任务、事件和状态可以在本体中建模，但执行顺序、超时、重试、分支和授权通常属于工作流和规则系统。

建议：将对象及关系、业务约束、流程编排与工具执行分层定义，并通过清晰接口关联。

反模式四：本体越大越好，试图一次性覆盖整个企业

全企业概念盘点可能很有价值，但若没有明确问题和责任边界，术语争议和跨团队协调会迅速增加。模型越大，版本变更的影响范围也越难控制。

此为临时预览链接，将在短期内失效

本自主与变更评审比一次性追求“完整本体”更重要。

九、把建模方法落到一份可以评审的产物

本体设计不应只留下一个图文件。第一版模型至少应包含以下内容：

1. **范围说明书**：目标、使用者、建模边界、不在范围内的内容。
2. **能力问题清单**：本体必须支持回答的问题及其验收标准。
3. **规范术语表**：名称、定义、别名、例子、反例、来源和责任人。
4. **核心类与关系表**：类的含义、属性、关系方向和必要约束。
5. **数据映射表**：领域概念与数据库字段、API、事件或文档之间的对应关系。
6. **验证用例**：代表性正常案例、异常案例和边界案例。
7. **开放问题与决策记录**：仍有争议的定义、采取的约定、决策人和后续复核时间。
8. **版本与变更记录**：何时修改了什么概念、为什么修改、影响哪些查询、Agent 和工作流。

这种产物让业务专家可以核对业务含义，架构师可以检查模型边界，开发人员可以实现数据映射，AI 工程团队也可以把能力问题转化为自动化评估用例。

一份轻量评审清单

在进入实现前，团队可以逐项检查：

- 本体的目标、用户与边界是否明确?
- 是否有具体、可回答、可测试的能力问题?
- 核心概念是否有稳定定义、唯一标识和例子?
- 同名异义、异名同义和概念重叠是否处理?
- 类、属性、关系、状态、事件和动作是否区分清楚?
- 是否只把当前用途所需的概念纳入范围?
- 是否评估过复用已有标准或本体?
- 模块边界、外部依赖和责任人是否清晰?
- 是否准备了正常、异常和边界测试?
- 模型中的关键规则是否映射到真正执行的系统门禁?
- 是否定义版本管理、变更审批和向后兼容策略?

如果多数问题仍无法回答，就不必急着进入 OWL 或图数据库实现。先把业务定义和范围谈清楚，通常更节省后续成本。

此为临时预览链接，将在短期内失效



【图 4｜本体设计评审与验收检查表】

十、让本体成为可验证的工程资产，而不是一张漂亮的图

本体建模最重要的判断，不是类和关系有多少，而是它是否对真实问题有帮助，是否能被不同团队一致理解，以及业务变化时是否能够安全演进。

一个精简的领域本体，如果能准确回答几个高价值问题，能映射到真实的数据源，能通过异常案例检验，并能被 Agent 和工程工作流可靠使用，就可能比覆盖全企业、却没人维护的大模型更有价值。

因此，我建议把建模过程当成一个循环，而不是一次性设计任务：

范围界定 → 能力问题 → 候选概念与关系 → 数据映射 → 验证 → 评审 → 迭代。

本体不是模型团队单独完成的产物，而是业务专家、数据工程师、架构师、安全人员和 AI 工程师共同维护的语义资产。

> **先明确要解决的问题，再决定描述世界的粒度；先定义可以验证的语义，再把它交给软件和 Agent 使用。**

系列预告

下一篇是第 07 篇：**《本体自动化构建工具全景：从概念发现到知识抽取，怎么选？》**

我们将比较自动化本体草拟、基于本体的知识抽取、语义层与知识图谱构建等不同工具路线，讨论哪些工作可以交给 AI 加速，哪些环节仍需要领域专家决策，以及如何评价自动化构建的质量。

此为临时预览链接，将在短期内失效

1. Stanford University — Ontology Development 101: A Guide to Creating Your First Ontology
2. Grüninger & Fox — The Role of Competency Questions in Enterprise Engineering
3. Halper et al. — Guidelines for the reuse of ontology content
4. W3C — OWL 2 Web Ontology Language Primer
5. W3C — Shapes Constraint Language (SHACL)

收录于 本体论在AI工程中的实践
作者提示: 内容由AI生成